Cabal iUSD Vault

Security Review

January 7, 2026 - January 12, 2026

Contents

1. About SBSecurity 3

2. Disclaimer 3

3. Risk classification 3

4. Executive Summary 4

5. Findings 5

Low severity 6

[L-01] dust asset accumulating in totalUnclaimedAssets 6
[L-02] maxDeposit should round up for max vault utilization 7
[L-03] vault is not entirely ERC4626 compliant 8
[L-04] withdrawal estimation functions must check for active assets staleness 9
[L-05] Off-by-one can lock entire batch shares 11
[L-06] Decrementing activeAssets can Reduce it below seedAmount without validation 13
[L-07] Protocol can lose fee due to unclaimed Assets 16
[I-01] Wrong comment in requestWithdraw 17
[I-02] CabalVault::beforeWithdraw internal function can be safely removed 17
[I-03] Some unused Errors can be safely removed 17
[G-01] public visibility used on functions, that are called only externally. 18

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter @Slavcheww.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.

4. Executive Summary

A time-boxed security review of the **ZeroXSigmoid/iUSD_SB_FINAL** repository was conducted by **SBSecurity**. The review was performed by a team of 3 security researchers, who identified **11 issues** in total.

Overview

Project	Cabal iUSD Vault
Commit Hash	f51a8b79e026cc9e338d842a8caf498fcede2daf
Resolution	242b445a878077c3ecb038f7ec6d7a44f5f3e145
Timeline	January 7, 2026 - January 12, 2026

Scope

CabalVault.sol

Post Audit Condition

The codebase is well-structured and follows solid development practices. 0 Critical/High/Medium severity issues were found. Additional audits would be a bonus, but are not mandatory.

5. Findings

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low/Info Risk	10
Gas Optimization	1

ID	Title	Severity	Resolution
[L-01]	dust asset accumulating in `totalUnclaimedAssets`	Low	Acknowledged
[L-02]	`maxDeposit` should round up for max vault utilization	Low	Acknowledged
[L-03]	vault is not entirely `ERC4626` compliant	Low	**Fixed**
[L-04]	withdrawal estimation functions must check for active assets staleness	Low	**Fixed**
[L-05]	Off-by-one can lock entire batch shares	Low	**Fixed**
[L-06]	Decrementing `activeAssets` can Reduce it below `seedAmount` without validation	Low	**Fixed**
[L-07]	Protocol can lose `fee` due to unclaimed Assets	Low	**Fixed**
[I-01]	Wrong comment in `requestWithdraw`	Info	**Fixed**
[I-02]	`CabalVault::beforeWithdraw` internal function can be safely removed	Info	Acknowledged
[I-03]	Some unused Errors can be safely removed	Info	Acknowledged
[G-01]	`public` visibility used on functions, that are called only externally.	Gas	**Fixed**

Low severity

[L-01] dust asset accumulating in `totalUnclaimedAssets`

Severity: Low Risk

Description: `totalUnclaimedAssets` in `_finalizeBatch` is based on `actualPulled`, but each individual `claim` is `user shares * batchClaimRatio`, the sum of all tokens actually claimed will be less than the cumulative vault assets for this batch due to the rounding that will happen in `claim`:

```
uint256 claimable = (userShares * batchClaimRatio[batch]) / 1e18;
```

Impact: Because of each individual claim being susceptible to rounding, dust will accumulate in the vault under the `totalUnclaimedAssets`. These tokens will be forever locked in the vault, as users and managers cannot withdraw them.

Recommendation: Track total unclaimed assets per batch (another variable you use only for that sole purpose) and harvest them in `pauseAndWithdrawToSelf`. It's important to clear them after the current batch concludes, but before the new one is finalized, not to include them in the calculations.

Resolution: Acknowledged

[L-02] `maxDeposit` should round up for max vault utilization

Severity: Low Risk

Description: Since `maxDeposit` is used to estimate the maximal number of assets that the vault would accept with regards all the caps. Currently, it reuses the `_convertToAssets` function that is used when calculating the assets to transfer out from the vault, hence it **rounds down** the assets users will get.

```
function maxDeposit(address) public view override returns (uint256) {
    // If the vault is shutdown, no one can deposit
    if (isShutdown) return 0;

    // If the sharesupplycap is the max number possible (no limit)
    // The user can practically deposit infinite
    uint256 _cap = shareSupplyCap;

    if (_cap == type(uint256).max) return type(uint256).max;

    // Else get the total current assets and supply and calculate
    // the delta from the max allowed
    uint256 _totalAssets = _totalAssetsHelper();
    uint256 _totalSupply = totalSupply();

    // If we're already at the cap, user can't deposit anything
    if (_totalSupply >= _cap) return 0;

    uint256 shareDelta = shareSupplyCap - _totalSupply;
    if (_totalSupply == 0 || _totalAssets == 0) return shareDelta;

    return _convertToAssets(shareDelta, _totalAssets, _totalSupply);//QA: this should round up
}
```

```
// Called when the user wants to REDEEM, so we round DOWN in assets they get
function _convertToAssets(uint256 shares, uint256 _totalAssets, uint256 _totalSupply)
    internal
    pure
    returns (uint256 assets)
{
    if (_totalSupply == 0) return shares;
    assets = shares.mulDiv(_totalAssets, _totalSupply, Math.Rounding.Floor);
}
```

However, for entries, i.e. how much users can deposit in the vault, it should round up instead.

Impact: Quite negligible impact, resulting in 1 wei off, reported from the `maxDeposit` function, which used by users to calculate their entry value, will leave the vault below the cap with 1 wei.

Recommendation: Consider doing the shares-to-asset conversion in the function itself by rounding up for a maximal vault capacity utilization.

Resolution: Acknowledged

[L-03] vault is not entirely ERC4626 compliant

Severity: Low Risk

Description: According to the EIP, the following mismatches are currently presented that can either create inconsistencies and integrator's code unreliable:

- `maxDeposit` doesn't return 0 when the vault is paused.

  > *MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily) it MUST return 0.*

- `previewDeposit` must revert on conditions that would cause `deposit` to revert as well (excluding the deposit limits)

  > *MUST NOT revert due to vault specific user/global limits. MAY revert due to other conditions that would also cause `deposit` to revert.*
  >
  > Conditions, causing `deposit` to revert, which `previewDeposit` doesn't:
  >
  > 1. Vault not initialized (`whenInitialized`)
  > 2. Active assets update is stale (`notStale`)
  > 3. Vault is in shutdown state (`_checkIfShutdown`)
  > 4. Vault is paused (`_checkIfPaused`)

- `previewMint` isn't overriden, but should be to revert, because `mint` reverts.

  > *MUST NOT revert due to vault specific user/global limits. MAY revert due to other conditions that would also cause `mint` to revert.*

- `previewWithdraw` must revert, because `withdraw` reverts

  > *MUST NOT revert due to vault specific user/global limits. MAY revert due to other conditions that would also cause `withdraw` to revert.*
  >
  > This one is pretty controversial, because `requestWithdrawal` is replacement of the `withdraw` function, but no instant withdrawal happens, hence the most appropriate way is to have a non-overriden preview for the withdrawal

Recommendation:

- `maxDeposit` - add check to return 0, when vault is paused
- `previewDeposit` - invoke the `whenInitialized`, `notStale` modifiers and `beforeDeposit` function
- `previewMint` - override and revert
- `previewWithdraw` - override and revert

Resolution: Fixed - `previewDeposit`, `maxDeposit`

[L-04] withdrawal estimation functions must check for active assets staleness

Severity: Low Risk

Description: `maxWithdrawRequestable`, `getSharePrice`, `getUserState` should ensure the reported active assets are not stale.

```
function maxWithdrawRequestable(address owner) public view returns (uint256) {
    // Get how many shares this user has
    uint256 ownerShares = balanceOf(owner);
    if (ownerShares == 0) return 0;

    uint256 _totalAssets = _totalAssetsHelper();
    uint256 _totalSupply = totalSupply();
    if (_totalSupply == 0 || _totalAssets == 0) return 0;

    // Calculate how many assets in iUSD this user is entitled to
    uint256 ownerAssets = _convertToAssets(ownerShares, _totalAssets, _totalSupply);

    return ownerAssets;
}

function getSharePrice() public view returns (uint256 price) {
    uint256 _totalAssets = _totalAssetsHelper();
    uint256 _totalSupply = totalSupply();
    if (_totalSupply == 0) return 1e18;
    return (_totalAssets * 1e18) / _totalSupply;
}

function getUserState(address user) external view returns (
    uint256 shares,
    uint256 assets,
    uint256 pendingShares,
    uint256 pendingBatch,
    bool batchIsFinalized,
    uint256 claimableAmount,
    uint256 maxDepositable,
    uint256 maxWithdrawable
) {
    shares = balanceOf(user);

    uint256 _totalAssets = _totalAssetsHelper();
    uint256 _totalSupply = totalSupply();
    assets = _convertToAssets(shares, _totalAssets, _totalSupply);

        ...MORE CODE

    maxDepositable = maxDeposit(user);
    maxWithdrawable = maxWithdrawRequestable(user);
}
```

Otherwise, the output from these function will be wrong as it won't be calculating the current performance of the vault.

Impact: No direct fund loss from these view functions alone. However, they can return stale share price/user asset estimates, which may mislead UIs, bots, and off-chain accounting systems. If strategy value decreases (strategy losses/manual manager operations) but accounting hasn't been updated yet, getSharePrice, maxWithdrawRequestable, and getUserState.assets can overstate user value. Once refreshed, values can drop, leading to misquotes and poor decisions.

This behavior can also occur in situations that are not strictly considered stale. However, the impact is exacerbated once the staleness threshold is exceeded, indicating prolonged inactivity by the manager in maintaining vault state correctness.

Recommendation: In `getSharePrice` and `maxWithdrawRequestable` invoke the `notStale` modifier.

Resolution: Fixed

[L-05] Off-by-one can lock entire batch shares

Severity: Low Risk

Description: In `_finalizeBatch`, require statement for the balance invariant accepts finalizations, where the manager haven't forwarded any tokens:

```
/// @dev Internal implementation of batch finalization
/// @param skipValidation If true, skips the 80% minimum ratio check
function _finalizeBatch(bool skipValidation) internal {
    // Ensure pullFromAdapters was called first (prevents admin error)
    // Should always be true since whenPaused is true
    if (!pullInitiated) revert PullNotInitiated();

    if (totalPendingShares == 0) revert NoPendingWithdrawals();

    // Calculate delta (Physical assets received)
    uint256 currentBalance = IERC20(asset()).balanceOf(address(this));

    // Protect against weird edge case where balance decreased (shouldn't happen if paused)
    require(currentBalance >= vaultBalanceBeforePull, "Balance decreased!");//@audit accepts ==

    uint256 actualPulled = currentBalance - vaultBalanceBeforePull;

    // NEW MATH: Assets per share
    // How much actual cash did we get for every share waiting in the queue?
    // 1e18 for precision - we divide by this later when the user claims
    uint256 assetsPerShare = (actualPulled * 1e18) / totalPendingShares;
```

As it can be seen, the scenario where `currentBalance == vaultBalanceBeforePull` is accepted.

`vaultBalanceBeforePull` is the last documented balance when the manager paused the vault for batch finalization.

Knowing that we can see that `assetsPerShare` will become 0, because of `actualPulled = 0`. This in combination with `skipValidation == true`, that bypasses the following invariant:

```
if (!skipValidation) {
    // Calculate expected share price at time of pause
    // Use vaultBalanceBeforePull + activeAssets since funds were in transit
    uint256 totalAssetsAtPause = vaultBalanceBeforePull + activeAssets;
    if (totalAssetsAtPause > totalUnclaimedAssets) {
        totalAssetsAtPause = totalAssetsAtPause - totalUnclaimedAssets;
    } else {
        totalAssetsAtPause = 0;
    }

    // Expected value per share based on current share price
    uint256 expectedPerShare = (totalAssetsAtPause * 1e18) / totalSupply();

    // Minimum acceptable: (100% - maxSlippageBps) of expected
    uint256 minAcceptable = (expectedPerShare * (BPS_DENOMINATOR - maxSlippageBps)) / BPS_DENOMINATOR;

    if (assetsPerShare < minAcceptable) revert InsufficientPull(assetsPerShare);
}
```

Will assign `batchClaimRatio == 0`, pricing all the shares for this batch worthless.

Impact: Shares for current batch will be locked in the Vault and users with pending withdrawals will lose their funds forever. Refunding and cancelations aren't possible after batch is finalized.

Recommendation: Modify `_finalizeBatch` to revert when no assets were forwarded:

```
/// @dev Internal implementation of batch finalization
  /// @param skipValidation If true, skips the 80% minimum ratio check
  function _finalizeBatch(bool skipValidation) internal {
      // Ensure pullFromAdapters was called first (prevents admin error)
      // Should always be true since whenPaused is true
      if (!pullInitiated) revert PullNotInitiated();

      if (totalPendingShares == 0) revert NoPendingWithdrawals();

      // Calculate delta (Physical assets received)
      uint256 currentBalance = IERC20(asset()).balanceOf(address(this));

      // Protect against weird edge case where balance decreased (shouldn't happen if paused)
-       require(currentBalance >= vaultBalanceBeforePull, "Balance decreased!");//@audit accepts ==
+       require(currentBalance > vaultBalanceBeforePull, "Balance decreased!");

      uint256 actualPulled = currentBalance - vaultBalanceBeforePull;

      // NEW MATH: Assets per share
      // How much actual cash did we get for every share waiting in the queue?
      // 1e18 for precision - we divide by this later when the user claims
      uint256 assetsPerShare = (actualPulled * 1e18) / totalPendingShares;
```

Resolution: Fixed

[L-06] Decrementing `activeAssets` can Reduce it below `seedAmount` without validation

Severity: Low Risk

Description: The CabalVault is initialized with a `seedAmount` of iUSD, and an equal amount of shares is minted to the `DEAD_ADDRESS`.

During the deposit flow, when a Manager calls `pauseAndWithdrawToSelf()`, the seedAmount worth of shares is also withdrawn and bridged to the Echelon chain via the Initia front-end. The Manager then calls `updateActiveAssets()`, which includes these `seedAmount` assets, causing them to be counted as `activeAssets`.

When funds are pulled back from Echelon and `finalizeBatch()` is called, `activeAssets` is reduced by `actualPulled` without any validation to ensure `newActiveAssets >= seedAmount`.

`_finalizeBatch()` snippet:

```
if (activeAssets >= actualPulled) {
    activeAssets -= actualPulled;  <@---- activeAssets Var is decreased without validation
}
```

Afterwards, the Manager calls `updateActiveAssets(uint256 newActiveAssets)` which also lacks the validation to ensure `newActiveAssets >= seedAmount`. If the entire Echelon balance is pulled (including accrued `seedAmount` mistaking it for accrued yield) then activeAssets can be set to 0.

This means:

1. With `activeAssets == 0`, `_totalAssetsHelper()` returns `0`, causing all deposits to revert due to division by zero.

2. Once totalAssets == 0, an attacker can deposit 1 wei of iUSD and mint the entire seedAmount of shares:

$$\text{shares} = \frac{\text{assets} \times \text{totalSupply}}{\text{totalAssets}} = \frac{1 \text{ wei} \times 50e18}{1 \text{ wei}} = 50e18$$

Impact:

1. Legitimate user deposits will revert, preventing normal depositors from interacting with the vault.

2. Any user can transfer as little as 1 wei of iUSD to the vault and call `deposit()` to mint the entire `seedAmount` of shares, or deposit more to push the total share supply up to the vault's maximum supply cap.

Proof of Concept:

1. Paste the Following PoC in `CabalVault.deposit.t.sol` and run it using `forge test --mt test_NoSeedAmount_Check_in_Finalize -vv`

```
function test_NoSeedAmount_Check_in_Finalize() public {
    uint256 depositAmount = 1_000e18;

    // Fund user FIRST (before expectEmit, since mint emits Transfer)
    _fundUser(user1, depositAmount + 2);

    // User 1 deposits tokens
    vm.prank(user1);
    uint256 sharesReceived = vault.deposit(depositAmount, user1);

    console.log("Shares received 1 : %e", sharesReceived);

    //New Manager is Set
    address Manager = makeAddr("Manager");
    vm.prank(owner);
    vault.setManager(Manager, true);

    //Manager withdraws funds from vault (which includes Seed Amount)
    vm.startPrank(Manager);
    vault.pauseAndWithdrawToSelf();
    vault.setPause(false);
    vm.stopPrank();

    //Active Assets Updated 1000e18 + 55e18(seedAmount)
    vm.prank(owner);
    vault.updateActiveAssets(1055e18);

    //User 2 deposits tokens
    _fundUser(user2, depositAmount);

    vm.prank(user2);
    uint256 sharesReceived2 = vault.deposit(depositAmount, user2);

    console.log("Shares received 2 : %e", sharesReceived2);

    vm.startPrank(Manager);
    vault.pauseAndWithdrawToSelf();
    vault.setPause(false);
    vm.stopPrank();

    //Active Assets Updated
    vm.startPrank(owner);
    vault.updateActiveAssets(2000e18);
    vault.updateActiveAssets(2055e18);
    vm.stopPrank();

    //Both Users Request Redeem
    vm.prank(user1);
    vault.requestRedeem(1000e18);

    vm.prank(user2);
    vault.requestRedeem(1000e18);

    //Admin Accidentally transfers whole echelon Amount thinking that 55e18 is accrued yield
    _processWithdrawalBatch(2055e18);

    //Both users claim (uncommenting these claims below will also make the PoC to pass)
    // vm.prank(user1);
    // vault.claim();

    // vm.prank(user2);
    // vault.claim();

    console.log("user 1 balance  : %e", iUSD.balanceOf(user1));
    console.log("user 2 balance  : %e", iUSD.balanceOf(user2));
```

```
//Now User 1 transfers 1 wei iUSD to vault and deposits 1 wei
    // and receives 5.5e19 shares
    vm.startPrank(user1);
    iUSD.transfer(address(vault), 1);
    sharesReceived = vault.deposit(1, user1);
    vm.stopPrank();

    console.log("Shares received 1 : %e", sharesReceived);
    assertEq(sharesReceived, 5.5e19);
}
```

Recommendation: Make the following changes in to mitigate the issue:

1. Make sure to store the `seedAmount` in an immutable state-variable in the initialize function

2. In `CabalVault::_finalizeBatch` function:

```
function _finalizeBatch(bool skipValidation) internal {

    ...
    ..
    .
        if (activeAssets >= actualPulled) {
        activeAssets -= actualPulled;
    } else {
        activeAssets = 0;
    }

+   require(activeAssets >= seedAmount, "seed Amount cannot be withdrawn");
}
```

3. In `CabalVault::updateActiveAssets` function:

```
function updateActiveAssets(uint256 newActiveAssets) external onlyOwnerOrManager {
          // Sanity check: don't allow wild swings (unless first update)
    if (activeAssets > 0) {
        uint256 maxIncrease = activeAssets * (10000 + MAX_CHANGE_BPS) / 10000;
        uint256 maxDecrease = activeAssets * (10000 - MAX_CHANGE_BPS) / 10000;
        if (newActiveAssets > maxIncrease || newActiveAssets < maxDecrease) {
            revert UpdateTooLarge();
        }
    }

+   require(newActiveAssets >= seedAmount, "seed Amount cannot be withdrawn");
    activeAssets = newActiveAssets;
    lastUpdateTimestamp = block.timestamp;

    emit ActiveAssetsUpdated(newActiveAssets, block.timestamp);
}
```

The managers must make sure that this `seedAmount` of asset tokens always stays in circulation preferably leaving them in the Echelon as they will be accounted as `activeAssets` to mitigate the issue

Resolution: Fixed

[L-07] Protocol can lose `fee` due to unclaimed Assets

Severity: Low Risk

Description: During the withdrawal flow, the `Manager` calls `finalizeBatch()`, which makes the pulled assets available for the `user` to claim. After the withdrawal process is finalized, the user can call `claim()` to receive these assets.

The issue here is that the `fee` is transferred to the protocol only when the user calls `claim()`. As a result, if assets remain unclaimed, the protocol does not receive the associated `fee`. Additionally, delayed claims cause the protocol to receive the `fee` much later than intended, leading to deferred or potentially lost fee revenue.

Impact: Collecting the `fee` in the `claim` function leads to inconsistent, delayed or potential loss of protocol `fee`.

Recommendation: It is recommended to calculate and collect the `fee` for the entire batch at once in the `_finalizeBatch()` function itself (i.e., `fee = (actualPulled * feeBps) / BPS_DENOMINATOR`). The fee calculation in the `claim()` function can then be removed and the claimable amount can be transferred directly to the caller.

Resolution: Fixed

[I-01] Wrong comment in `requestWithdraw`

Severity: Information Risk

Description: `requestWithdraw` doesn't immediately burns shares, but transfers them to the vault. However, there's a wrong comment, indicating shares have to be burnt:

```
// Request a withdrawal in assets. Burns shares immediately.
    // Adds to the current batch for processing.
    function requestWithdraw(uint256 assets) external nonReentrant notPaused whenInitialized notStale {
    //CHECK: should we have notStale here?
        if (assets == 0) revert ZeroAssets();
```

Recommendation: Replace the wrong comment with *"Transfer shares to Vault, serving as an escrow"*.

Resolution: Fixed

[I-02] `CabalVault::beforeWithdraw` internal function can be safely removed

Severity: Information Risk

Description: The `CabalVault::beforeWithdraw` function is not used anywhere within the contract. In the parent `ERC4626` contract, it is only invoked by the `redeem()` and `withdraw()` functions, both of which are overridden and intentionally reverted in CabalVault. As a result, `beforeWithdraw()` is never reachable and can be safely removed.

Recommendation: `beforeWithdraw()` can be safely removed

Resolution: Acknowledged

[I-03] Some unused Errors can be safely removed

Severity: Information Risk

Description: `CabalVault.sol` has some unused errors

```
error SlippageExceeded(address adapter, uint256 expected, uint256 actual);
error AlreadyInitialized();
error InvalidSeedRecipient();
error InsufficientSeedAssets();
```

which can be safely removed

Recommendation: Unused errors can be safely removed

Resolution: Acknowledged

[G-01] `public` visibility used on functions, that are called only externally.

Severity: Gas Optimization

Description: functions that aren't being invoked in the vault, but are used only externally are marked as public, resulting in a high gas consumption, because when there's a `public` visibility modifier Solidity copies the entire `calldata` into memory.

This doesn't happen when `external` is being used.

Recommendation: Change the visibility modifier in the following functions:

1. `setFeeBps`
2. `setFeeRecipient`
3. `increaseShareSupplyCap`
4. `decreaseShareSupplyCap`
5. `getSharePrice`

Resolution: Fixed